

ソフトウェアサプライチェーンセキュリティ分析のための GSN 自動生成手法と適用事例

太田 雄貴[†]

Automated GSN Generation Method for Software Supply Chain Security Analysis and Its Application Examples

Yuuki OHTA[†]

あらまし 近年、SolarWinds 事件や Top.gg 事件に代表されるソフトウェアサプライチェーン攻撃が急増し、甚大な被害をもたらしている。本研究では、複雑なサプライチェーンのセキュリティリスクを構造的に可視化し、対策の妥当性を評価する手法を提案する。具体的には、サプライチェーンセキュリティに特化した ASiRA モデルを入力とし、安全性の論理的保証ケースを記述する GSN (Goal Structuring Notation) を自動生成する手法およびツールを開発した。本手法は、コンポーネント間の依存関係を ASiRA モデルに基づき解析して GSN を生成する。また、各要素のセキュリティ要件に対する対策の有無を確認し、無防備な箇所を抽出する。提案手法を SolarWinds および Top.gg 事件の事例に適用した結果、大規模な GSN を即座に出力し、攻撃の侵入経路となった箇所の不備を具体的に特定できることを確認した。本研究成果は、複雑なシステムにおけるサプライチェーンセキュリティの評価プロセスの効率化と信頼性向上に寄与するものである。

キーワード ソフトウェアサプライチェーンセキュリティ, GSN, ASiRA モデル, 自動生成

1. 序 論

ソフトウェアサプライチェーンセキュリティとは、ソフトウェアの開発から配布、そして利用に至るまでの全ての段階において、その安全性を確保するための取り組みを指す。近年、情報セキュリティ分野ではサプライチェーンセキュリティに焦点が当てられている。これは、SolarWinds 事件に代表されるように、ソフトウェアサプライチェーンへの攻撃によって甚大な被害が生じる事例が増えているためである [1]。

このような背景を受け、米国政府はソフトウェアサプライチェーンセキュリティを強化するため SBOM の義務付けを行う大統領令を発し [2]、サイバーセキュリティ戦略においてソフトウェアサプライチェーンに関する戦略を立てており [3]、日本政府もソフトウェアサプライチェーンセキュリティを意識したサイバーセキュリティ戦略 [4] を発するなど、ソフトウェアサ

プライチェーンセキュリティの強化に積極的に取り組んでいる。

本研究では、ソフトウェアサプライチェーンセキュリティを強化するため、システムのサプライチェーンが抱えるセキュリティリスクを可視化して対策を容易にすることを目指す。

ソフトウェアサプライチェーンのセキュリティを効果的に評価するために、GSN(Goal Structuring Notation) [5] に注目した。GSN とはシステムが安全であることを論理的かつ視覚的にわかりやすく示すための手法の一つである。システムとそれを構成するサプライチェーンを GSN を用いて表すことにより、セキュリティ分析を実行できると考えた。また、GSN は大規模なシステムになればなるほど大きな図になってしまう、手書きで書くのは大きな手間になってしまうため自動で生成して利用することを考え、以下の 3 点を実行した。

- **GSN 自動生成手法の考案:** 後述のサプライチェーンセキュリティモデルである ASiRA モデル [1] から GSN を自動生成する手法を考案した。

[†] 福井大学大学院工学研究科
Graduate School of Engineering, University of Fukui
DOI:10.14923/transinfj.??????????

- **GSN 自動生成ツールの開発:** 考案した手法をもとに、GSN を自動生成するツールを作成した。

- **実際の攻撃事例に対する有効性の確認:** 実際の事例に対し、自動生成した GSN を用いてセキュリティが十分かを示した。

本論文の構成は以下の通りである。第 2 章では先行研究について述べ、過去の研究やセキュリティモデルと本研究の差異を示す。第 3 章・第 4 章では本研究で使用する AStRA モデルと GSN について述べる。第 5 章では第 3 章・第 4 章で説明した AStRA モデルから GSN を自動生成し、生成した GSN を利用してセキュリティ分析を行う手法について述べる。第 6 章・第 7 章では提案手法を用い、SolarWinds 事件と Top.gg への攻撃について事例分析を行い、本手法の有効性を検証する。第 8 章では第 6 章、第 7 章で行った事例分析によって得られた知見について述べる。第 9 章では本論文のまとめを行う。

2. 先行研究

ソフトウェアサプライチェーンセキュリティの領域では、脅威分析やセキュリティ保証のためのモデルやフレームワークが先行研究として存在する。ここでは、SLSA [6] と STRIDE モデル [7]、AttackTree [8]、AStRA モデル、GSN について述べる。

2.1 2.1 SLSA について

SLSA とは、Google が提唱したソフトウェアサプライチェーンの完全性とセキュリティを保証するためのフレームワークである。これは、ソフトウェア成果物が信頼できるサプライチェーンから提供されたものであることを検証可能にするための、一連のセキュリティ要件と検証可能な基準を定義する。SLSA は、ソフトウェアのビルドプロセス、ソースコードの管理、依存関係の扱いなど、サプライチェーンの各側面におけるセキュリティの成熟度をレベル 0 からレベル 3 までの 4 段階で示す。レベルが高いほどより高度なセキュリティであることを示す。4 段階の詳細は以下の通りである。

- **Build L0:** 何の対策もない状態。
- **Build L1:** パッケージの構築方法を示す来歴がある。人為的なミスを防ぐことができる。
- **Build L2:** ホスト型ビルドプラットフォームによって生成された署名済みの来歴がある。ビルド後の改ざんを防止できる。
- **Build L3:** 強化されたビルドプラットフォーム

がある。ビルド中の改ざんを防止できる。

SLSA はサプライチェーン全体やコンポーネントに対してセキュリティのレベルに応じて分類をしているだけであるが、本研究ではサプライチェーンのコンポーネントに応じたセキュリティ対策や脆弱である理由などを同時に分析できるという相違点がある。

2.2 2.2 STRIDE モデルについて

STRIDE モデルとは、Microsoft が提唱する IT 全般でのシステム脅威を分析する手法である。システムを構成要素として分類し、それぞれにデータフローを作って分析を行う。STRIDE とは以下の 6 つのセキュリティの要素の頭文字である。

- **Spoofing of user identity (偽装):** なりすまし。
- **Tampering (改竄):** データやコードの書き換え。
- **Repudiation (否認):** 操作履歴が残らないなど、ユーザの行動が追跡できない状態。
- **Information disclosure (情報漏洩):** 機密や個人情報などの流出。
- **Denial of service (サービス拒否):** サービスを利用できない状態。
- **Elevation of privilege (権限昇格):** 本来は利用できない管理者権限など、上位の権限を不正に得ること。

このセキュリティ目標や脅威などを 6 つの要素に分類を行い、それぞれに防御策を設定している。

2.3 2.3 AttackTree について

AttackTree とは、古くからあるセキュリティ分析手法である。木構造をもち、大本となる根 Node に攻撃者の最終目標を置き、葉 Node に攻撃者が実行する具体的手段を配置する。攻撃者の視点に立って必要な攻撃を分解し、どのセキュリティを強化すべきかを分析する手法となっている。

STRIDE モデルや AttackTree は汎用的なセキュリティモデルでありソフトウェアサプライチェーンセキュリティに特化したモデルではないが、本研究はソフトウェアサプライチェーンセキュリティに特化しているという点で相違点がある。

2.4 2.4 本研究で使用する AStRA モデルと GSN について

AStRA モデルは、他のセキュリティモデルと異なりソフトウェアサプライチェーンセキュリティに特化したセキュリティモデルである。詳細は第 3 章にて説明する。

GSN とは、「このシステムは安全である」という大き

な目標 (Goal) に対し、それを裏付ける戦略 (Strategy) や証拠 (Evidence) をピラミッド状の図でつなぎ、論理的に説明するための手法である。詳細は第 4 章にて説明する。

本研究では、ソフトウェアサプライチェーンセキュリティに特化したセキュリティモデルである AStRA モデルと、論理的に安全性を示すことができる GSN を使用し、論理的かつわかりやすくソフトウェアサプライチェーンのセキュリティリスクを可視化する。

3. AStRA モデルについて

AStRA モデルとは、ソフトウェアサプライチェーンセキュリティに特化したセキュリティモデルの一つである。あるシステムについて、非巡回有向グラフ (DAG) を使用して表現する。システムの各コンポーネントを以下の 5 つに分類し、セキュリティ分析を行う。

- **Principal (主体)**：ソフトウェアサプライチェーンに関与する個人・団体・組織。
- **Artifact (成果物)**：ソフトウェアの作成、構成、評価のために必要なデジタル情報の基本単位。
- **Step (工程)**：Principal が Artifact に対して、リソースを用いて実行する操作や処理単位。
- **Resources (資源)**：Artifact の作成・構成・評価に使用される、独自のサプライチェーンを持っているコンポーネント。
- **Topology (全体構造)**：構成要素とその因果関係を含めた全体像。

AStRA モデルは、Principal が Resource を使う、Resource が Step を実行する、Step が Artifact を生産または消費するという 3 つのルールで構成されている。本研究では Topology を除く 4 つの要素を使用する。

AStRA モデルでは各要素にセキュリティ要件が設定されており、Principal には 5 つの、Artifact、Step、Resource の 3 要素には 3 つのセキュリティ要件が存在する。本論文で登場するセキュリティ要件は AStRA モデルでのセキュリティ要件に準拠する。

図 3.1 (本文参照) は 4 つの Node で構成されており、Principal が Resource を使用し、Resource が Step を実行し、Step が Artifact を生産する様子を示している。

次の図 3.2 (本文参照) は、後述する事例である SolarWinds 事件が発生したシステムでの実際の AStRA モデルである。この図において、青色は Principal を、緑色は Resource を、黄色は Artifact を、赤色は Step

をそれぞれ指す。これは、SolarWinds 社のビルドシステムで、Node 数 22 と大規模なシステムであることがわかる。一番右に位置する Binary Repository が下層のコンポーネントを集めて構成されている様子を示している。

4. GSN について

GSN は、あるシステムや製品が安全であることを論理的かつ構造的に示すための記法である。本研究では、Goal、Strategy、Evidence、Undeveloped の 4 つの要素を用いる。

- **Goal**: 達成すべきセキュリティ目標や主張。
- **Strategy**: Goal を示すための手順や方針。
- **Evidence**: Goal が妥当であることを示す根拠。
- **Undeveloped**: 現時点で Evidence がないことを表す。

GSN は最上位の Goal を TopGoal といい、この TopGoal を論証することを目的とする。上位の Goal を Strategy を通して下位の SubGoal に分解し、SubGoal に対してさらに分解が可能であれば Strategy を通してより下位の SubGoal に分解を行う。それ以上 Goal を要素ごとに分解できない場合、その Goal を最下層の Goal とし、その Goal を満たす根拠があれば Evidence を、ない場合は Undeveloped とする。

図 4.1 (本文参照) に GSN の例を挙げる。この GSN 図は、TopGoal となる Goal を Strategy で SubGoal1 と SubGoal2 に分解する。この SubGoal について、SubGoal1 は Evidence が存在するため、この Goal は証明されているといえる。SubGoal2 は Undeveloped となっており、この Goal が証明されていないことを示している。

次の図 4.2 (本文参照) は、後述する事例である SolarWinds 事件が発生したシステムの GSN の一部である。この GSN 図は Tekton standard CI/CD pipeline が安全であるという主張を TopGoal に置き、それを下位の Goal に分解している。下層には Evidence がなく Undeveloped となっている Goal があるが、結節点となる Goal の Evidence が完全であれば、その Goal の主張は保証されていると言える。

5. 提案手法

5.1 5.1 本手法の概要

本手法では、AStRA モデルの入力データから GSN を自動生成し、それを用いてシステムのセキュリティ

妥当性を評価する。具体的には、コンポーネントの構成情報を入力として、論理構造の構築と Evidence の紐付けを自動で行い、未検討事項を抽出する。分析においては、生成された GSN のボトムアップな論理構造に基づき、各 Node に提示された対策が下位の階層からの攻撃をどの程度阻止・検出できるかを評価することで、システムの安全性を確保するための要件を明確化する。

5.2 5.2 GSN の自動生成プロセス

まず、開発した GSN 自動生成ツールについて説明する。本ツールは、Python を用いて開発した。AStRA モデルから得られたコンポーネント情報を入力として GSN を構築する。入力データは以下の 2 つの形式で与えられる。

Node (辞書型のリスト) : 各 Node には「Node 名」「AStRA モデルにおける分類」「Evidence (存在する場合)」が含まれる。Evidence は辞書型で構成され、具体的な「セキュリティ要件」とそれに対応する「セキュリティ対策」を保持する。

Edge (タプル型のリスト) : グラフの辺を定義し、「始点 Node 名」「終点 Node 名」「ラベル」の順で情報を保持する。

ツールの処理フローは以下の通りである。

1. **グラフの Node の作成と処理順序の決定:** 入力された Node と Edge のリストに基づいて、有向グラフを構築する。構築したグラフに対してトポロジカルソートを実行する。これにより、システムの依存関係の上流から下流へと順番に GSNNode を並べる。ここで作成した順序に基づき再帰的に 2 と 3 の処理を行う。

2. **グラフ Node から GSN の Goal を生成:** 各 Node に対して、[Node 名] は安全である、という Goal と Strategy を作成する。このとき、この Goal に対応するセキュリティ要件を示す SubGoal を生成し、Evidence があれば Evidence を、なければ Undeveloped を付加する。Undeveloped となる Goal はログにまとめられる。

3. **GSN の Node 同士を連結して GSN を構築:** Top-Goal から、各 Goal を依存関係に従って Strategy を用いて分解する SubGoal として、下流の Goal を付加するという形で 1 つの GSN を構築する。

4. **GSN と不足するセキュリティ要件の表示:** 完成した GSN と、Undeveloped を収集したログをもとに作成した不足するセキュリティ要件のリストを表示する。

本ツールの主要な関数を以下に示す。このアルゴリズムは、入力された AStRA モデルのグラフ構造を走

査し、再帰的に GSN 要素を構築するものである。

- **generate_security_requirement_goal:** この関数中で Goal と Evidence の紐づけを行う。Evidence がなければ Undeveloped を付与し、ログに記録する。
- **make_goal:** この関数中で generate_security_requirement_goal 関数を呼び出し、Node の Goal を生成する。生成した Goal に対し Strategy を生成し、依存関係に応じて SubGoal に下位 Goal とセキュリティ要件の SubGoal を付与する。
- **generate_gsn_goals_from_graph:** 入力にした Node と Edge のリストを受け取り、ソートを行い順序を決定する。この順序に基づき Make_Goal を呼び出し、GSN 全体を構築する。
- **display_missing_evidence_from_log:** Undeveloped となった Goal について、セキュリティ対策が必要なことを通知する。
- **Main:** 入力となる Node と Edge のリストを読み込んで generate_gsn_goals_from_graph を呼び出し、得られた GSN を出力する。

この GSN と Undeveloped リストを確認することで、対象システムがどの要件を満たせば安全性を確保できるかを明確に特定することが可能となる。

5.3 5.3 GSN を使用したセキュリティ分析手法

生成された GSN を用いた分析は、以下の 2 つのルールに基づいて行う。基本的には、下位の Goal が満たされていると、対応する上位 Goal の安全性が保障されるというボトムアップの論理構造をとる。Goal に対し、下位に位置する SubGoal と Goal 自身のセキュリティ要件を示す SubGoal がともに満たされていると、その Goal は満たされているため安全であると主張する。

これに加え、本研究では GSN の各結節点となる Goal において、Evidence として提示されたセキュリティ対策が「完璧である」と判断される場合、それより下層の Goal に Undeveloped があるとしても、結節点の Goal からつながる下層から発生する攻撃はその箇所検出または阻止が可能であるとして分析を行う。

図 5.2 (本文参照) のように、SubGoal のセキュリティ要件を満たす Evidence がなくボトムアップのルールでは安全が保障されているとはいえない。しかし、Goal 自身のセキュリティ要件に対する Evidence が完全である場合、これより下位の攻撃はここで検知・阻止できるためシステムの上層に届かないので、安全であると主張できる。

Evidence に使用されている「P1:○○」、「S1:○○」

などとして出てくるものは対応する AStRA モデルでのセキュリティ要件を示している。

6. SolarWinds 事件での事例分析

具体的なサプライチェーン攻撃の事例として SolarWinds 事件を取り上げ、この事件の AStRA モデルから GSN を生成し、その結果からセキュリティ対策について考える。

まず、SolarWinds 事件とは、2020 年に米国で発生した大規模なサプライチェーン攻撃である。攻撃者はまず CI/CD 管理者のアカウントを乗っ取り、このアカウントを通して SolarWinds 社のビルドシステムにマルウェアを混入した。このマルウェアは顧客がアップデートを行うことで SolarWinds 社のシステムを利用していた顧客の端末にインストールされ、攻撃者はこのマルウェアを通じて情報の窃取などを行った。この事件発生時のセキュリティ対策とこの事件発生後のセキュリティ対策について、それぞれ GSN を生成し比較を行った。

次に、SolarWinds 社のセキュリティ対策について事件前と事件後で比較する。事件前は有効となる対策が何も取られていなかったが、事件後は管理者などのアカウントは多要素認証を必要とし、使用時に必要なアクセス権限のみを渡すという形をとるようにした。また、侵入を受けたビルドシステムは使用のたびに破棄されて再構築され、さらにすべての操作が記録されるようになり、コードが悪用された場合の証明書の破棄などの対策がとられた。

SolarWinds 事件における AStRA モデルを図 6.1 (本文参照) に示す。この SolarWinds 事件のシステムを分析した AStRA モデルと事件発生前のセキュリティについて、GSN を生成した。AStRA モデルでの Node 数は 22、Edge 数は 27 と大きなモデルを入力とした。

その結果、セキュリティ要件をのぞく Subgoal の総数は 44 と巨大な図となった。全体図は付録 A を参照すること。その一部を図 6.2 (本文参照) に示す。これは生成した GSN 図から SolarWinds 事件で攻撃を受けた箇所を切り取ったもので、侵入経路となった Principal である CI/CD controller の項目には安全性を保証する根拠となる Evidence がなく、安全な状態ではなかったと言える。

不足していた Evidence の出力の一部を図 6.3 (本文参照) に示す。このように、Evidence の添付がなくセキュリティ要件を満たす根拠がないものについては、

必要なセキュリティに関する取り組みを表示する。

事件後にセキュリティ対策がとられたものに対してと同様に GSN を生成した。全体図は付録 A を参照すること。その結果の一部を図 6.4 (本文参照) に示す。図 6.4 より Principal である CI/CD controller と Step である Build にセキュリティ対策が追加されたことで、SolarWinds 事件と同様の攻撃を受けても検知して対処することができるようになった。

これらの結果から、AStRA モデルから GSN を自動生成することで、手書きで書くには困難な規模の GSN を短時間で出力することができ、その GSN を元にシステムのサプライチェーンの弱点を特定し、不足するセキュリティに関する取り組みを確認することができた。また、SolarWinds 事件前後での GSN を比較することで、SolarWinds 事件と同様の攻撃に対してシステムの対策が行えていることも確認した。

7. Top.gg 事件での事例分析

次に、2024 年 3 月に報告された Discord 用プラットフォームである Top.gg を対象としたサプライチェーン攻撃を事例として、提案手法の有効性を検証した。

Top.gg は、Discord の Bot やサーバーを検索するための大規模なプラットフォームである。攻撃者はまず、Top.gg の GitHub リポジトリの管理権限を持つユーザーを標的とし、ブラウザのセッションクッキーを窃取することで、二段階認証を回避してアカウントを乗っ取った。

次に、攻撃者は Python の公式パッケージリポジトリである PyPI に酷似したドメインを用意し、広く利用されているライブラリの一つである Colorama の偽パッケージを配置した。この偽パッケージには、悪意のあるコードが埋め込まれていた。その後、乗っ取った管理者アカウントを用いて Top.gg の公式リポジトリ内の依存関係ファイルを書き換え、偽パッケージが自動的にダウンロードされるよう変更を加えた。これにより、当該リポジトリを利用する他の開発者や利用者がインストールを実行した際、マルウェアが配布され、情報の窃取が行われる結果となった。

前述の攻撃事例に基づき構築した AStRA モデルを図 7.1 (本文参照) に示す。本モデルは、攻撃対象となった開発者、リポジトリ、外部ライブラリなど 8 個の Node で構成されている。

Top.gg で実行されていたセキュリティ対策に関して、同社からの声明がないため詳細は不明であるが、

GitHub を利用していたことから GitHub で実行されているアカウントの二段階認証が設定されていたと推測される。しかし、この攻撃は認証済みの状態を保持するセッションクッキーを標的としたため、既存のセキュリティ対策が機能しなかった点が挙げられる。

構築した AStRA モデルの Node と Edge を入力し、GSN を自動生成した。その結果、セキュリティ要件を除く SubGoal の総数は 11 となった。生成された GSN の全体像を図 7.2（本文参照）に示す。

生成された GSN を用いた分析の結果、侵入経路となった Principal である Top.gg Maintainer の Node において、二段階認証の実施がされており Evidence があるものの、セッション管理の脆弱性をカバーする対策が不足しており、今回の攻撃に対しての適切な Evidence がなかったといえる。同様の攻撃に対しては侵入経路となった Principal のセキュリティをより強固にすること、侵入を受けても結節点となる Step である Commit において、不正な操作を防止できるセキュリティ対策を行うという 2 点があれば、Principal で必要な Evidence と結節点となる Step における Evidence が GSN に追加され、防止できたと考えられる。

8. 得られた知見

以上 2 つの事例より、Node 数が大きく、手書きで書くには困難な大きさの GSN を正しく自動生成できることを確認した。このことから、実際の複雑なシステムに対して本研究の GSN 生成手法は実用性を持つことを確認した。

SolarWinds 事件の事例から、事件前のシステムの GSN では侵入経路となった項目が Undeveloped となっており、視覚的にセキュリティ対策がなされていないことが明示された。また、新たに取り組まれたセキュリティ対策により Undeveloped となっていた箇所の Evidence が埋められ、同様の攻撃に対して対処が可能になったことを GSN から確認した。また、不足する Evidence のリストからシステムで全くセキュリティ対策がない箇所を確認することができた。Top.gg 事件の事例においても、同様に GSN からセキュリティの不十分を確認した。

これらより、本研究のセキュリティ分析手法は GSN と不足する Evidence のリストを用いることで視覚的にシステム全体が安全であるかの論証を行い、特定の攻撃に対するセキュリティ対策の不備を具体的に特定できる手法であることが分かった。

しかし、本手法では AStRA モデルでの 5 つの要素のうち、全体構造を示す Topology の要素を GSN に反映できないためサプライチェーンの構造そのものに問題がある場合の議論が現状不可能である。また、セキュリティ要件の設定や GSN の生成に AStRA モデルを利用しているが、Top.gg の事例のような特殊な攻撃方法に対して有効なセキュリティ要件を提供できないという問題点も発見した。

9. ま と め

本研究では、複雑化するソフトウェアサプライチェーンにおけるセキュリティリスクを可視化し、論理的な安全性を保証するための GSN を自動生成する手法の提案と、GSN を利用したセキュリティ分析手法について実際に発生したソフトウェアサプライチェーン攻撃の事例に適用しその有効性を検証した。

まず、サプライチェーンの構成要素を Principal、Artifact、Step、Resource に分類する AStRA モデルと、安全性の主張を構造化する GSN を組み合わせることで、システムの依存関係に従い GSN を自動生成する手法を考案した。次に、Python を用いた自動生成ツールを開発し、トポロジカルソートによる解析を通して、大規模なシステムに対しても一貫性のある GSN を短時間で出力することを可能にした。また、セキュリティ対策を GSN の Evidence として利用することで、どのコンポーネントにセキュリティ対策が行われていないかを容易に確認できるようにした。

本手法の評価として、SolarWinds 事件と Top.gg 事件の事例分析を行った。SolarWinds 事件での事例分析では、Node 数 22 のモデルから 40 以上の Goal を持つ GSN を生成し、GSN を自動生成することの有効性を示した。また、事件前後の対策状況を比較することで、同様の攻撃を阻止できるようになったことを論理的に示した。Top.gg 事件の分析では、既存の二段階認証という Evidence だけではセッション管理の脆弱性を防げないことを可視化し、どの結節点に対策を追加すべきかを具体的に特定した。これにより、本手法が単なる GSN 図の生成のみに留まらず、実用的なセキュリティ分析の指針を提供できることを実証した。

今後の展望として、以下の課題が挙げられる。第一に、AStRA モデルの全体構造を示す Topology の GSN への反映である。現在の実装では個々のコンポーネントの安全性については論証可能であるが、サプライチェーン全体の問題点の分析には、Topology の統合が

不可欠である。第二に、セキュリティ要件の動的な更新である。Top.gg 事件のようなセッションクッキーを標的とするといった特殊な攻撃手法に対し、最新の脅威情報を反映し、適切なセキュリティ要件を提示する仕組みの構築が求められる。

10. 事例 SolarWinds 事件

具体的なサプライチェーン攻撃の事例として SolarWinds 事件を取り上げ、この事件の AStRA モデルから GSN を生成し、その結果からセキュリティ対策について考える。(中略) その一部を図 1 に示す。

図 3：SolarWinds 事件発生前の GSN（一部）

図 1 SolarWinds 事件発生前の GSN（一部）

11. ま と め

本研究では、AStRA モデルと GSN を使ってセキュリティリスクの可視化とそれを満たす取り組みを示した。

文 献

- [1]
- [2]

(xxxx 年 xx 月 xx 日受付)

太田 雄貴

福井大学 大学院工学研究科 知識社会基礎工学専攻 情報工学コース 博士前期課程 1 年。サプライチェーンセキュリティの研究に従事。

Abstract Software supply chain security refers to efforts to ensure safety at all stages from development to distribution and use. In this study, we aim to visualize security risks and facilitate countermeasures by automatically generating GSN from the AStRA model.

Key words Software Supply Chain Security, GSN, AStRA Model, Automated Generation